

浙江大学



分布式智能物资管理系统

姓 名 黄鹤翔

学 号 3230106231

院 系 信息与工程学院

指导老师 杜歆

摘要

本项目围绕实验室物资借还场景，设计并实现了一套分布式智能物资管理系统。系统将树莓派侧图像采集、服务端人脸识别、YOLO 物资检测、TCP 协议通信、出入库差分记录以及本地 Web 看板整合在一起，形成“身份确认-物资识别-自动登记-结果追溯”的完整闭环。与只做单张图检测的原型相比，本项目更关注真实使用流程中的可部署性与可追溯性：客户端不再依赖 ROS2，服务端内部采用 ROS2 节点解耦识别任务，跨设备之间只通过轻量 TCP 协议交互；库存记录先落到 CSV，再由本地看板汇总展示，既降低了课程项目的实现复杂度，也保留了后续扩展到数据库系统的空间。实验结果表明，当前三类物资数据集上模型在第 31 轮训练时取得了 87.74% 的 mAP50 和 76.15% 的 mAP50-95，能够支撑课程项目阶段的闭环验证。报告重点从系统背景、运行流程、关键技术选择与结果展示四个角度展开，对比分析各模块方案的取舍与优势。

关键词：嵌入式系统；分布式协同；YOLO 目标检测；人脸识别；TCP 通信；库存看板

目录

摘要	1
1 项目背景与需求分析	3
2 系统总体设计	3
2.1 架构划分	3
2.2 模块分工	4
3 系统运行逻辑	5
4 关键技术与实现分析	7
4.1 图像采集与客户端状态机	7
4.2 TCP 协议与分布式解耦	7
4.3 物资识别模块 (YOLO)	8
4.4 人脸识别模块	9
4.5 库存差分、CSV 记录与 Web 看板	9
5 结果展示与项目亮点	10
5.1 训练结果与模型表现	10
5.2 单图推理与闭环验证	12
5.3 项目亮点归纳	12
6 总结与展望	13
参考资料	14

1 项目背景与需求分析

实验室和小型仓储场景中的物资管理，表面上是“记账”问题，实际牵涉到身份确认、数量核对、责任追踪和后续查询等多个环节。传统做法通常依赖纸质登记表、微信群接龙或者电子表格手工录入，这类方式在物资种类少、人员固定时尚可维持，一旦进入多人共享、频繁借还的使用环境，就容易暴露出三类问题。

其一，记录与操作脱节。物资是现场取走的，但登记往往发生在事后，容易出现“先拿后补”“记了一半忘了提交”等情况。其二，责任链不完整。若不先确认操作者身份，后续只能看到某个数量发生变化，却难以准确追溯是谁执行了这次入库或出库。其三，系统部署门槛高。若将检测、识别、记录、展示全部堆到一台设备上，计算资源和开发复杂度都会迅速抬升；若直接采用跨机器 ROS2 通信，又会给客户端环境配置带来额外负担。

基于以上问题，本项目没有把目标限定为“识别图片中的几个物资”，而是把重点放在闭环流程的构建上。具体而言，系统需要满足以下四点要求：

1. 在用户执行出入库动作之前，先完成身份识别，避免匿名操作。
2. 在操作前后各获取一次物资图像，通过数量差分自动生成库存变化记录，而非要求用户手工输入数量。
3. 将客户端采集与服务端识别解耦，保证树莓派等轻量设备也能稳定接入。
4. 提供一个本地可视化看板，能够查看当前库存状态、最近操作人员和历史变化轨迹。

从目前仓库的真实实现看，系统已经具备上述闭环能力。其中，物资检测模型当前先以 cboard、dmj4310、m3508 三类物资为主完成验证；而记录、看板和协议层则按照通用物资条目来设计，便于后续扩展到更多类别。这样的推进顺序比较符合课程项目的节奏：先跑通系统主链路，再逐步扩展识别范围，而不是一开始就追求大而全。

2 系统总体设计

2.1 架构划分

本项目采用典型的客户端-服务端协同结构，其中客户端负责本地交互与抓拍，服务端负责识别、汇总和展示。这样做的关键原因在于，树莓派侧更适合承担摄像头控制和界面交互，而模型推理、人脸编码库和库存看板更适合集中放在计算资源更充足的服务端主机上。

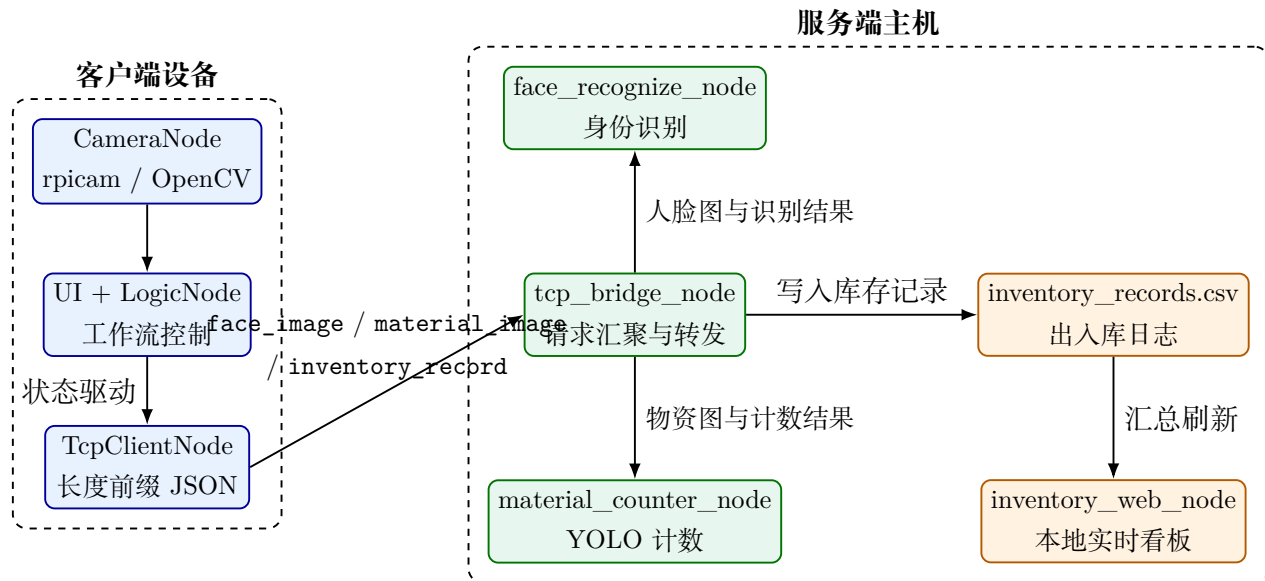


图 1: 系统总体结构图

图 1 展示了本项目的核心分工。客户端侧由 CameraNode、LogicNode 和 TcpClientNode 组成，其中 LogicNode 负责整条业务流程的状态推进；服务端侧保留 ROS2 包结构，但 ROS2 只在服务端内部承担节点解耦的作用，人脸识别节点和物资识别节点通过 topic 与 tcp_bridge_node 协同，跨设备则完全走 TCP。这样既保留了 ROS2 对模块划分的便利，也避免了在客户端部署 ROS2 带来的学习和配置成本。

2.2 模块分工

为了让各模块职责更加清晰，系统可以进一步拆成表 1 所示的几个功能层次。

表 1: 系统主要模块与职责

层次	核心模块	主要职责
客户端采集层	相机采集模块	统一封装摄像头抓拍接口，优先调用 <code>rpicam-still</code> ，必要时退回 <code>OpenCV</code> 。
客户端控制层	状态机与 UI 模块	维护 workflow 状态，控制人脸识别重试、物资前后拍摄、差分计算和记录提交。
网络通信层	TCP 客户端 / TCP 网桥	通过长度前缀 JSON 协议在客户端与服务端之间传输图像请求和库存记录。
服务端识别层	人脸识别节点 / 物资识别节点	分别完成人脸识别与 YOLO 物资检测，并将结果封装为 JSON 返回。
数据展示层	CSV 日志 / Web 看板	将变更记录落盘为 <code>inventory_records.csv</code> ，再汇总成库存总览和详情看板。

从整体设计看，这套结构并不追求一次性引入最复杂的基础设施，而是强调“每一层都有明确输入输出”。课程项目阶段更看重系统是否稳定闭环，因此本项目优先用轻量但透明的方式完成联调，例如库存记录先写 CSV 而不是直接上数据库，看板默认只绑定 127.0.0.1 而不开放外网。这样的设计虽然保守，却更有利于在有限时间内把问题定位清楚。

3 系统运行逻辑

系统主流程由客户端状态机驱动，服务端识别节点根据请求类型提供结果，库存记录在差分阶段才真正生成。其运行逻辑如图 2 所示。

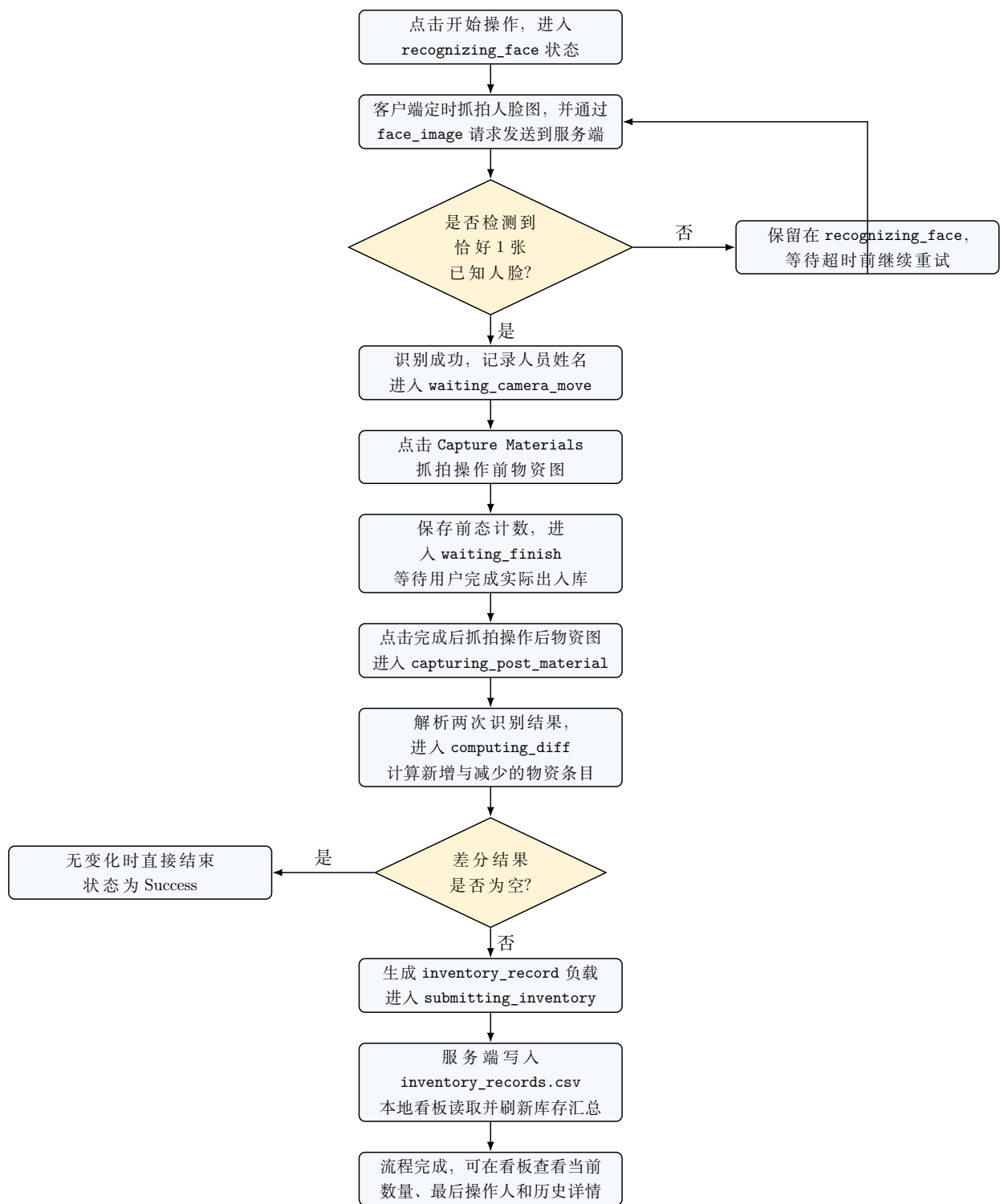


图 2: 系统运行主流程图

与常见“持续视频流实时盘点”思路不同，本项目把一次完整的库存变更分成了两次物资拍摄和一次差分计算。这样做的优势在于，系统只需要关心操作前后数量是否发生变化，而不必在

用户移动物资、遮挡镜头或操作过程中持续做复杂判断。具体来看，这条流程的关键触发点有三个：

1. 只有在人脸识别返回“恰好一张已知人脸”时，流程才允许进入物资拍摄阶段；
2. 操作前物资图改为人工点击 `Capture Materials` 触发，而不是依赖自动稳定判定；
3. 只有在后态计数和前态计数求差之后，系统才真正生成入库或出库记录并写入 CSV。

这三个触发点对应的都是课程项目里最常见的失败源：身份不清、抓拍时机错误、记录与实际动作不同步。把它们显式做成状态转换条件，比把所有逻辑藏在一段连续视频处理流程里更容易调试，也更便于后期复盘。

4 关键技术与实现分析

4.1 图像采集与客户端状态机

原理：客户端需要在资源有限的设备上稳定获取图像，同时维持一个清晰可控的工作流。对树莓派这类平台而言，图像采集本身就会受到驱动、后端和桌面环境的影响，因此采集链路必须足够简单；而库存业务又具有明确阶段性，适合用状态机而非散乱回调来组织。

具体实现：当前实现中，`CameraNode` 优先选择 `rpikam-still` 抓拍 JPEG 输出，如果运行环境没有该工具，再自动退回 `OpenCV`。UI 层保留了 `Start`、`Capture Materials` 和 `Finish` 三个关键按钮，`LogicNode` 通过 `recognizing_face`、`waiting_camera_move`、`waiting_finish`、`computing_diff` 等状态推进完整流程。尤其值得注意的是，项目后期把“自动稳定后抓拍前态物资”改成了手动按钮触发，这一变化直接减少了误抓拍。

方案对比：如果采用连续视频自动抓拍方案，理论上交互更少，但在树莓派环境中容易受到帧率波动、相机移动和现场遮挡影响，稳定性判断稍有偏差就会在错误时刻截取前态图像。当前手动触发方案虽然多了一次点击，却能显著降低误判和返工成本，更符合课程项目先求稳后求自动化的思路。

本节亮点：这部分最有价值的地方并不是“功能更多”，而是把系统中最容易失控的抓拍阶段收敛成了一个用户可理解、开发者也容易调试的流程。对嵌入式项目而言，稳定往往比炫目的自动化更重要。

4.2 TCP 协议与分布式解耦

原理：客户端与服务端分布在不同设备上时，需要一个足够轻量且可持续复用连接的通信机制。由于这里传输的是图像和结构化记录，使用“长度前缀 + JSON”的 TCP 协议可以避免粘包拆包问题，同时保持报文可读性。

具体实现：客户端 `TcpClientNode` 会把图像编码为 JPEG 后再转成 Base64，再按请求类型发送给服务端。当前定义了三类核心请求：

- `face_image`：上传人脸图，用于身份识别；
- `material_image`：上传物资图，用于 YOLO 检测与计数；
- `inventory_record`：上传最终库存变化记录。

服务端 `tcp_bridge_node` 负责统一接收请求、转发到相应识别节点，并把结果再次封装为 JSON 响应。通信层还加入了超时和重连逻辑：当请求失败时，客户端会关闭旧连接并尝试重新建立 socket，这比每次都从头初始化整套流程更稳妥。

```
1 {
2   "type": "inventory_record",
3   "request_id": "req-002",
4   "payload": {
5     "time": "2026-04-25T21:30:00+08:00",
6     "person": "huanghexiang",
7     "action": "出库",
8     "items": [{ "name": "cboard", "quantity": 2 }]
9   }
10 }
```

方案对比：与“跨机器直接跑 ROS2”相比，当前方案不要求客户端具备 ROS2 环境，也不需要处理 DDS 配置、节点发现等额外问题；与“所有接口都改成 HTTP”相比，TCP 长连接对频繁小请求的处理更直接，而且与当前客户端状态机实现更贴合。因此，这种分布式解耦方式在异构设备协同时更容易落地。

本节亮点：客户端已经被改造成独立 Python 包，意味着以后即使替换 UI 框架或部署平台，只要保留这三类 TCP 请求，服务端识别链路就能继续复用，系统的扩展成本会明显降低。

4.3 物资识别模块（YOLO）

原理：物资识别的核心任务是对单张图像中的目标进行检测和分类，再按类别统计数量。YOLO 检测模型能够在一次前向推理中同时完成目标定位和类别判断，因此适合在复杂背景下进行多目标计数。

具体实现：项目训练脚本默认以 `yolo11n.pt` 小模型为起点，在自建数据集上进行微调。当前数据集规模为训练集 37 张、验证集 19 张、测试集 8 张，类别包括 `cboard`、`dmj4310` 和 `m3508` 三类。服务端 `material_counter_node` 负责加载 `weights/materials_yolo/best.pt`，对输入图

像执行推理，并返回各类别计数字典。单图预测示例中，系统已经能够输出 `cboard: 2、m3508: 1` 这样的计数结果。

方案对比：如果改用传统颜色阈值、边缘轮廓或模板匹配方法，在目标姿态变化、部分遮挡或背景杂乱时很容易失效，而且每增加一种物资就要手工重写一套规则。YOLO 虽然需要标注数据和训练过程，但一旦模型成型，对视角变化和复杂背景的适应能力明显更强，更适合后续扩展类别。

本节亮点：当前模型并没有一味追求大规模类别，而是先在三类典型物资上完成闭环验证。这样的策略使系统层与算法层能够同步推进：模型精度达到可用后，库存记录和看板能力立刻就能接上，而不是等到“算法彻底成熟”之后再重写整套系统。

4.4 人脸识别模块

原理：人脸识别部分的目标不是做复杂的人脸属性分析，而是在工作流开始前确认“当前操作者是否属于已登记人员”。因此该模块更适合采用已知人脸编码库加距离匹配的方式，而不是引入额外硬件或复杂的多阶段认证机制。

具体实现：服务端 `face_recognize_node` 在启动时会读取 `server/assets/known_face/` 中的样本图片，为每个已知人员生成人脸编码。运行过程中，节点使用 `face_recognition` 库检测人脸位置、提取编码，并与已知编码库比较欧氏距离。当前默认阈值为 `tolerance=0.45`，检测模型为 `hog`。客户端只接受“恰好 1 张已知人脸”的结果，多人同时入镜或只识别出 `unknown` 时，流程都会停留在重试状态。

方案对比：与手工输入姓名相比，人脸识别减少了人为漏填和冒名顶替的风险；与刷卡或指纹方案相比，它不需要额外硬件，且能直接复用现有摄像头采集链路。选择 `hog` 而不是更重的 `cnn` 方案，也是考虑到课程项目阶段更重视部署简洁和 CPU 侧响应速度。

本节亮点：人脸识别在本系统中承担的是“流程门控”而不是“炫技展示”。把身份确认放在物资识别之前，可以让后续每一条 CSV 记录天然带有责任归属，这一点对物资管理场景比单纯提升识别炫酷程度更有实际意义。

4.5 库存差分、CSV 记录与 Web 看板

原理：物资出入库本质上是一个“前后状态比较”问题。只要能得到操作前后两次识别结果，就可以通过计数字典求差，自动拆分出入库条目，再把结果落盘并可视化展示。

具体实现：客户端在两次物资识别之后调用差分逻辑，得到新增条目和减少条目。若差分为空，则直接判定本次流程无变化；若差分不为空，则构造 `inventory_record` 请求，并将 `action` 字段分别设置为“入库”或“出库”。服务端收到后写入 `server/assets/inventory_records.csv`，字段包括请求编号、记录时间、人员、动作、物资名和数量。本地 `inventory_web_node` 以 1 秒

刷新周期轮询 CSV，并在 127.0.0.1:8600 提供实时看板，用户可以查看当前库存数量、最近操作人、最近操作时间及某一物资的历史明细。

方案对比：直接手填 Excel 的方式虽然上手简单，但记录与现场操作之间没有自动关联，追溯性弱；若一开始就引入数据库、认证系统和复杂权限模型，又会明显抬高课程项目的实现难度。CSV 方案恰好位于两者之间：它足够轻量、可直接审计，也便于后续迁移到数据库。看板默认只在本机开放，不支持外网访问，这一设计同样是出于部署安全和调试便利的考虑。

本节亮点：差分记录和看板结合之后，系统不再停留在“检测出几个框”，而是变成了一个真正能被人使用的业务闭环。尤其是点开看板后能够直接看到某类物资的最近操作轨迹，这让系统的输出不再只是算法结果，而是可被管理者解释和追踪的数据。

5 结果展示与项目亮点

5.1 训练结果与模型表现

从训练日志来看，模型最优结果并不出现在最后一轮，而是在第 31 个 epoch 左右达到较好平衡，这一点与当前数据集规模较小、样本波动较大的实际情况是一致的。表 2 汇总了本次训练中较有代表性的指标。

表 2: YOLO 模型较优阶段指标（第 31 轮）

Epoch	Precision	Recall	mAP50	mAP50-95
31	0.8021	0.8036	0.8774	0.7615

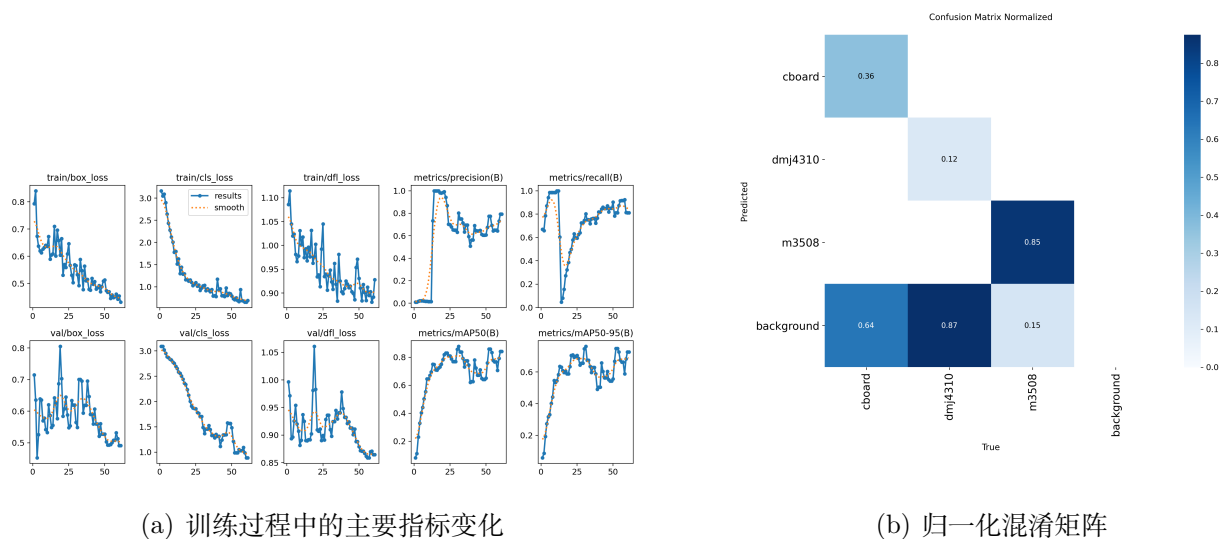


图 3: 模型训练结果与类别区分情况

图 3 左图展示了训练过程中损失、精度和召回率的整体变化趋势。可以看到，模型在中期已经具备较强的收敛能力，后续波动更多反映的是数据量有限带来的不稳定性，而不是模型完全不可用。右侧的归一化混淆矩阵则表明三类目标已经具备基本区分能力，当前结果足以支撑系统原型验证，但距离更大规模、更复杂场景下的稳定应用仍有提升空间。

5.2 单图推理与闭环验证

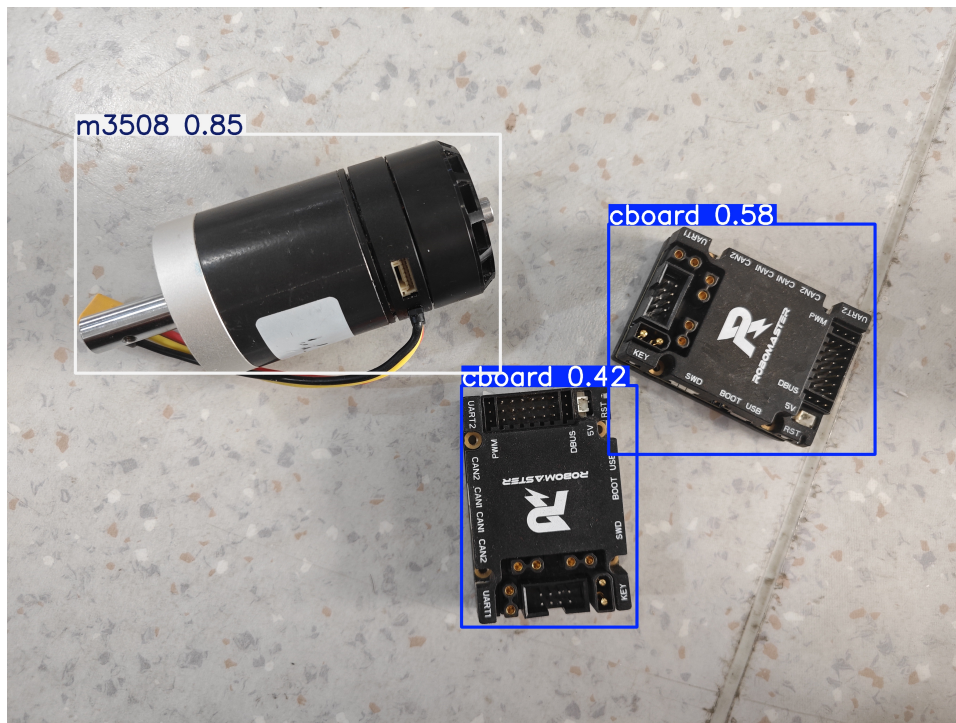


图 4: 单张测试图像的物资检测结果示例

图 4 给出了单张测试图像的推理结果。从输出文件可以看到，该样例最终计数为 cboard: 2、m3508: 1。这类计数结果在本系统中并不会停留在算法展示层，而是会被直接送入库存差分模块，与操作后的计数做比较，最后生成结构化的入库或出库记录。换句话说，检测结果不仅是“看起来识别对了”，而是能继续驱动后续业务逻辑。

5.3 项目亮点归纳

结合前文分析，可以把本项目当前阶段的亮点概括为以下四点。

1. **客户端脱离 ROS2，部署门槛更低。**客户端已经被重构为独立 Python 程序，适合在树莓派或普通 Linux 主机上直接运行，服务端则保留 ROS2 节点化优势，两者边界清晰。
2. **身份确认前置，责任链天然闭合。**系统在物资流程开始前就要求识别出唯一已知人脸，这比事后补填姓名的做法更可靠，也更便于追溯。
3. **前后帧差分代替人工登记。**本项目没有要求用户手工输入每种物资的数量，而是通过前后两次识别结果求差，这在体验上更接近真实管理场景。
4. **本地看板提高了结果可解释性。**记录不再只是躺在文件里，而是会以当前库存、最近操作人、详情时间线的形式持续展示，方便教师或同学直接观察系统效果。

从课程项目角度看，这几项亮点并不是互相孤立的。正因为前端采集、服务端识别、记录落盘和看板展示被串成了闭环，系统才摆脱了“单点算法演示”的局限，开始具备一个小型嵌入式应用应有的完整性。

6 总结与展望

本报告围绕一套分布式智能物资管理系统，梳理了其设计背景、运行逻辑、关键技术和实验结果。项目当前已经完成客户端采集、身份识别、物资识别、差分记录与 Web 看板展示的整条主链路，证明了“轻量客户端 + 集中式服务端 + 本地追溯看板”这一路线在课程项目中是可行的。更重要的是，这套系统并没有把重点放在单次识别炫技上，而是把识别能力放进了一个能真正完成登记和追踪的业务流程里。

当然，系统仍存在较明显的局限。当前检测模型只在三类物资上完成了验证，样本规模也较小；人脸库成员数量有限，尚未覆盖更复杂的光照和姿态变化；CSV 方案适合课程项目调试，但不适合高并发和多权限协同场景；操作前物资拍摄仍需人工点击，说明自动化程度还有进一步提升空间。

后续若继续迭代，本项目比较值得投入的方向主要有三点：一是扩充数据集规模和类别数量，使检测模型从“原型可用”走向“场景可用”；二是将 CSV 记录逐步替换为数据库和权限控制模块，支持更规范的库存管理；三是继续优化前态抓拍策略和端侧实时性，在保证稳定的前提下提升自动化程度。对课程项目而言，这样的演进路线既现实，也具备持续扩展的价值。

参考网站

- [1] 本项目代码仓库中的 README.md、client/README.md、server/README.md 与交接文档 docs/client_server_handoff_2026-04-27.md。
- [2] Ultralytics Docs. <https://docs.ultralytics.com/>
- [3] ROS 2 Humble Documentation. <https://docs.ros.org/en/humble/>
- [4] face_recognition Documentation. <https://face-recognition.readthedocs.io/en/latest/>